



1.2. Handling Unique and Duplicate Values

Previous Section:

 [1.1. Imputation of Missing Values](#)

Contents:

[1. What are Unique and Duplicate Values?](#)

[2. Working with Unique Values in Python](#)

[3. Working with Duplicate Values in Python](#)

[3.1. Detecting Duplicate Rows](#)

[3.2. Applying `duplicated\(\)` to Our Dataset](#)

[3.3. Removing Duplicate Rows](#)

[Summary](#)

[References](#)

By now, we've seen that missing values are common in almost every real-world dataset. Fortunately, they are usually easy to identify—Pandas can tell us exactly where they are. **Duplicate values** are a different story.

Sometimes duplicate records are obvious mistakes, such as the same customer being entered twice. But other times, duplicates are perfectly valid.

- For example, a sales database may contain multiple purchases made by the same customer, or a time-series dataset may record the same sensor every second.

Simply deleting duplicates without understanding the data can accidentally remove important information.

On the other hand, **unique values** are just as important. Knowing how many distinct values a feature contains helps us understand the data and decide how to process it.

- For example, before performing **label encoding**, we often need to know all possible categories that appear in a column.

In this section, we'll learn how to inspect both unique values and duplicate records using Pandas.

1. What are Unique and Duplicate Values?

A **unique value** is simply a distinct value that appears in a column.

- For example, if the **Major** column contains: `AI, AI, IT, CSC, AI, SW, IT`
 - Then the unique values are: `AI, IT, CSC, SW`

Although there are seven entries, there are only **four unique values**.

Knowing the unique values of a feature helps us understand the dataset and is particularly useful when working with **categorical variables**, feature engineering, and encoding techniques.

A **duplicate record** is a row that appears more than once in a dataset and usually represents the same real-world entity or event.

Duplicate records can occur for many reasons, including:

- Data collected from multiple sources.
- A user submitting the same form multiple times.
- Errors while merging or importing datasets.
- Accidental duplication during data entry.

Duplicate records can distort statistical analysis, inflate counts, and reduce the quality of machine learning models. However, not every duplicate is an error. Before removing duplicate rows, we should always understand **why** they exist and whether they represent legitimate repeated events.

2. Working with Unique Values in Python

Pandas provides two convenient methods for exploring unique values:

- `unique()` returns all distinct values in a column.

- `nunique()` returns the number of distinct values.

To display the unique values of every column, we can iterate through the DataFrame.

```
for column in df.columns:
    unique_values = df[column].unique()
    print(f"{column}:")
    print(unique_values)
    print(f"Number of unique values: {len(unique_values)}\n")
```

Since `unique()` returns a NumPy array, we can simply use `len()` to count the number of distinct values.

- If we're only interested in the count, Pandas provides a much simpler solution using `nunique()`.

```
print(df.nunique())
```

This prints the number of unique values for every column.

- We can also examine a single column.

```
print(df["Major"].unique())
print(df["Major"].nunique())
```

Sample output:

```
[nan 'SW' 'IT' 'CSC' 'AI']
```

```
4
```



If a DataFrame column has missing values. Then "NaN" sentinel **will also be included in the unique values**

These two methods are frequently used during **exploratory data analysis (EDA)** because they quickly reveal the possible categories in a feature, detect

unexpected values, and help determine whether a column is categorical, numerical, or even an identifier.

3. Working with Duplicate Values in Python

Pandas provides two useful methods for working with duplicate records:

- `uplicated()` identifies duplicate rows.
- `drop_duplicates()` removes duplicate rows.

Before removing duplicates, it is usually a good idea to **detect and inspect them first**. This helps us verify whether the duplicates are genuine mistakes or valid repeated records.

3.1. Detecting Duplicate Rows

The `uplicated()` method returns a Boolean Series:

- `True` means the row is considered a duplicate.
- `False` means the row is treated as the original occurrence.

By default, `uplicated()` uses `keep="first"`, meaning the first occurrence is kept while later duplicates are marked as `True`.

To better understand how the `keep` parameter works, consider the following dataset.

Name	Age	City
John	40	Bergen
Mary	30	Oslo
John	40	Stavanger
Sally	50	Oslo
Mary	30	Oslo
John	40	Stavanger
John	50	Stavanger
John	40	Stavanger

Notice that only the rows `(Mary, 30, Oslo)` and `(John, 40, Stavanger)` appear multiple times. The other rows are unique because at least one value is different.

- `keep=False`: Marks **every occurrence** of a duplicated row.

```
import pandas as pd

data = {
    "Name": ["John", "Mary", "John", "Sally", "Mary", "John", "John", "John"],
    "Age": [40, 30, 40, 50, 30, 40, 50, 40],
    "City": [
        "Bergen", "Oslo", "Stavanger", "Oslo",
        "Oslo", "Stavanger", "Stavanger", "Stavanger"
    ]
}

df_demo = pd.DataFrame(data)

print(df_demo.duplicated(keep=False))
```

Output:

```
0  False
1   True
2   True
3  False
4   True
5   True
6  False
7   True
dtype: bool
```

Every duplicated record is marked as `True`.

- `keep="first"` (**Default behavior in Pandas DataFrame**): This keeps the **first occurrence** and marks only the later duplicates.

```
print(df_demo.duplicated(keep="first"))
```

Output:

```
0  False
1  False
2  False
3  False
4   True
5   True
6  False
7   True
dtype: bool
```

- `keep="last"`: This keeps the **last occurrence** and marks the earlier duplicates.

```
print(df_demo.duplicated(keep="last"))
```

```
0  False
1   True
2   True
3  False
4  False
5   True
6  False
7  False
dtype: bool
```

3.2. Applying `duplicated()` to Our Dataset

Our synthetic student dataset was generated randomly, so complete duplicate rows are uncommon. To better demonstrate duplicate detection, we'll create a smaller DataFrame containing only the **Name** and **Major** columns.

```
# Add a new student name Kevin studying IT
df.loc[len(df_imputed)] = ["Kevin", 20, "IT", 3.5]

student_small = df[["Name", "Major"]]
print(student_small.head())
```

Note that if the dataset has missing values. It is suggested you impute them first

Now we can identify rows where **both columns match**.

```
duplicates = student_small[student_small.duplicated(keep=False)]
print(duplicates)
```

The duplicates should be (Kevin, IT) as we inserted this new student to showcase the method

Important: `duplicated()` compares the **entire row** (or the selected columns). A row is only considered a duplicate if **every compared value matches**. Having the same name alone does **not** make two rows duplicates if their majors are different.

For example:

Name	Major	Duplicate?
James	AI	✗
James	IT	✗
James	AI	✓

Only the first and third rows are duplicates because **both** the **Name** and **Major** columns are identical.

3.3. Removing Duplicate Rows

Once we've inspected the duplicates, removing them is straightforward using `drop_duplicates()`.

By default, Pandas keeps the **first occurrence** of each duplicated row.

```
student_unique = student_small.drop_duplicates()
print(student_unique.head())
```

- If we want to keep the last occurrence instead:

```
student_unique = student_small.drop_duplicates(keep="last")
```

- Or remove **all** duplicated rows completely:

```
student_unique = student_small.drop_duplicates(keep=False)
```

Summary

This section covers unique and duplicate values in data analysis and how to deal with them.

- **Unique values** are the distinct values that appear in a column. Pandas provides `unique()` to retrieve them and `nunique()` to count them.
- Inspecting unique values helps us understand categorical features and prepares the data for tasks such as label encoding and exploratory data analysis.
- **Duplicate rows** represent repeated records in a dataset, but not every duplicate is necessarily an error. Always understand why duplicates exist before removing them.
- `duplicated()` identifies duplicate rows without modifying the DataFrame.
- The `keep` parameter controls which occurrences are treated as duplicates:
 - `keep="first"` keeps the first occurrence.
 - `keep="last"` keeps the last occurrence.
 - `keep=False` marks every duplicated row.
- A row is considered a duplicate only if **all compared columns match**.
- `drop_duplicates()` removes duplicate rows and is a common data-cleaning operation before further analysis.

To practice further, complete Lab 01 of the Data Analytics Hands-On Lab.

 Lab 07: Data Analytics with Python

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.duplicated.html>

<https://pandas.pydata.org/docs/reference/api/pandas.unique.html>

<https://www.geeksforgeeks.org/python/pandas-dataframe-duplicated/>

<https://www.geeksforgeeks.org/python/get-unique-values-from-a-column-in-pandas-dataframe/>

Next Section:

 2.1. Data Length, Shape, and Size